MANUAL TÉCNICO Y ANÁLISIS DE DOCUMENTACIÓN

Guía de Trabajo con JPA en Spring Boot y Resumen de Documentos

PARTE 1: Paso a Paso — Inicialización y Desarrollo JPA en Spring Boot

Para trabajar con persistencia de datos relacionales mediante **JPA (Java Persistence API)** e **Hibernate** en un proyecto de Spring Boot, se requiere seguir un proceso metódico de configuración y desarrollo de componentes por capas.

1. Requisitos Previos y Dependencias (`pom.xml`)

Al crear el proyecto (vía Spring Initializr o Maven), es necesario añadir los starters y drivers correspondientes al archivo de configuración de dependencias `pom.xml`:

- `spring-boot-starter-data-jpa` : Incluye JPA, Hibernate como proveedor ORM por defecto, y la infraestructura de Spring Data.
- `spring-boot-starter-web` : Proporciona soporte para controladores REST, Spring MVC y el servidor embebido Tomcat.
- **Driver JDBC específico**: Por ejemplo, `mysql-connector-j` para MySQL (ámbito `runtime`).

```
<!-- Ejemplo de dependencias clave en pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

2. Configuración de la Conexión (`application.properties`)

Se debe modificar el archivo `src/main/resources/application.properties` para establecer las credenciales del motor de base de datos y el comportamiento de Hibernate:

```
# Parámetros de Conexión a la Base de Datos
spring.datasource.url=jdbc:mysql://localhost:3306/nombre_base_datos
spring.datasource.username=root
spring.datasource.password=secret
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Configuración de JPA e Hibernate
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Nota sobre `ddl-auto` *:* Los valores posibles son `none` (sin cambios), `validate` (comprueba esquema), `update` (actualiza tablas), `create` (recrea tablas al iniciar) y `create-drop` (elimina tablas al detener la app).

3. Clases e Interfaces a Crear (Arquitectura por Capas)

Para cada tabla o entidad del dominio se deben crear cuatro componentes estructurados:

A. Entidad JPA (`@Entity`)

Clase que representa una tabla de la base de datos. Requiere la anotación `@Entity`, una clave primaria `@Id` y un constructor vacío obligatorio.

```
@Entity
@Table(name = "articulos")
public class Articulo {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String titulo;

    @Lob
    @Column(nullable = false)
    private String contenido;

    @Column(nullable = false)
    private String autor;

    @Column(name = "fecha_publicacion", nullable = false)
    private LocalDate fechaPublicacion;

    public Articulo() {} // Constructor vacío obligatorio por especificación JPA
    // Getters y Setters...
}
```

B. Repositorio (`@Repository` / `JpaRepository`)

Interfaz que extiende `JpaRepository<Entidad, TipoId>`. Hereda operaciones CRUD, paginación y ordenación sin necesidad de escribir SQL explícito.

```
@Repository
public interface ArticuloRepository extends JpaRepository<Articulo, Long> {
    // Consultas derivadas automáticamente por nombre de método
    List<Articulo> findByAutor(String autor);
    List<Articulo> findByTituloContainingIgnoreCase(String palabra);
}
```

C. Capa de Servicio (`@Service`)

Clase anotada con `@Service` que centraliza la lógica de negocio, validaciones del dominio y la coordinación entre repositorios.

```
@Service
public class ArticuloService {
    private final ArticuloRepository articuloRepository;

    public ArticuloService(ArticuloRepository articuloRepository) {
        this.articuloRepository = articuloRepository;
    }

    public List<Articulo> listarTodos() {
        return articuloRepository.findAll();
    }

    public Articulo guardar(Articulo articulo) {
        return articuloRepository.save(articulo);
    }
}
```

D. Controlador (`@RestController` o `@Controller`)

Gestiona las peticiones HTTP entrantes, mapea parámetros y delega el procesamiento al servicio. En APIs REST devuelve JSON automáticamente mediante Jackson.

```
@RestController
@RequestMapping("/api/articulos")
public class ArticuloRestController {
    private final ArticuloService articuloService;

    public ArticuloRestController(ArticuloService articuloService) {
        this.articuloService = articuloService;
    }

    @GetMapping
    public ResponseEntity<List<Articulo>> listar() {
        return ResponseEntity.ok(articuloService.listarTodos());
    }
}
```

PARTE 2: Resumen Detallado del Documento 1

"Ciclo de Vida de una Petición HTTP en Spring Boot"

Este documento describe el trayecto completo que recorre una solicitud HTTP desde que es iniciada por un cliente (navegador, cliente REST, app móvil) hasta que alcanza la lógica interna de persistencia y regresa formalmente codificada en una respuesta HTTP.

```
Cliente → Tomcat → Filters → Spring Security → DispatcherServlet → HandlerMapping → Interceptors
              → Controller → Service → Repository → Hibernate/JDBC → Base de Datos
```

Desglose Paso a Paso por Componentes:

1. **Petición Cliente e Infraestructura Web Embebida (Tomcat):** Cuando se incluye `spring-boot-starter-web`, Spring Boot arranca un servidor web embebido (Apache Tomcat por defecto) en el puerto 8080. Tomcat recibe el flujo del socket de red y genera las instancias de la API Servlet: `HttpServletRequest` y `HttpServletResponse`. Cada petición es procesada habitualmente en un hilo de ejecución dedicado.
2. **Cadena de Filtros (Servlet Filters & Spring Security):** Los filtros actúan en la capa más externa (nivel Servlet).
   - **Filters:** Ejecutan lógica transversal (logging, CORS, modificación de cabeceras, compresión) invocando `filterChain.doFilter(request, response)`. Pueden abortar la petición prematuramente.
   - **Spring Security Filter Chain:** Intercepta la petición para validar credenciales o tokens JWT. Si la autenticación falla, responde `401 Unauthorized`; si carece de permisos, devuelve `403 Forbidden`, impidiendo la ejecución de controladores.
3. **El Front Controller (`DispatcherServlet`):** Componente central del patrón *Front Controller* de Spring MVC. Recibe todas las peticiones autorizadas y coordina el flujo hacia el manejador adecuado.
4. **Mapeo de Rutas (`HandlerMapping`):** Examina la URL y el método HTTP (GET, POST, PUT, DELETE) para identificar el método exacto del controlador mapeado mediante anotaciones (`@GetMapping`, `@PostMapping`, etc.).
5. **Interceptores de Spring MVC:** Componentes del entorno Spring MVC que ofrecen hooks de ejecución:
   - `preHandle()`: Se ejecuta antes del controlador (puede interrumpir el flujo).
   - `postHandle()`: Se ejecuta tras finalizar el controlador (antes de renderizar la vista).
   - `afterCompletion()`: Se ejecuta al completar el ciclo de procesamiento.
6. **Conversión de Parámetros y Validación:**
   - Spring convierte automáticamente tipos de datos en parámetros (ej. de cadena `"5"` a `Long 5` con `@PathVariable` o `@RequestParam`).
   - **`HttpMessageConverter` y Jackson:** El cuerpo JSON entrante (`@RequestBody`) se deserializa a objetos Java mediante Jackson.
   - **Validación (Jakarta Validation):** Si la petición usa `@Valid`, anotaciones como `@NotBlank` o `@Min` validan el DTO. Si falla la validación, se genera una respuesta de error `400 Bad Request` sin pasar al controlador.
7. **Capa Controlador, Servicio y Repositorio:**
   - **Controller:** Mapea y delega la lógica de negocio sin concentrar decisiones complejas.
   - **Service (`@Service`):** Contiene la lógica del dominio, control de transacciones y coordinación de varios repositorios.
   - **Repository (`@Repository`):** Abstrae el acceso a datos. Spring Data JPA utiliza Hibernate como proveedor ORM para traducir invocaciones de objetos en sentencias SQL, las cuales se ejecutan en la BD relacional (MySQL, PostgreSQL) mediante controladores JDBC de bajo nivel.
8. **Camino de Vuelta y Manejo de Excepciones:**
   - El resultado retornado por el repositorio sube al servicio y de este al controlador.
   - El objeto Java es serializado a JSON por Jackson mediante `HttpMessageConverter`.
   - El controlador emite un `ResponseEntity` estableciendo el cuerpo y los códigos de estado HTTP adecuados (`200 OK`, `201 Created`, `404 Not Found`).
   - **Gestión Global de Excepciones:** Mediante `@RestControllerAdvice` y `@ExceptionHandler`, las excepciones no capturadas se interceptan de forma centralizada, formateando respuestas JSON estándar de error.
   - La respuesta recorre de forma inversa Interceptores, Filtros y Tomcat hasta transmitirse al cliente.

PARTE 3: Resumen Detallado del Documento 2

"Acceso a Almacenes de Datos (JPA, Hibernate y ORM)"

El segundo documento se centra en los fundamentos del Mapeo Objeto-Relacional (ORM), la arquitectura de persistencia en Java/Spring Boot y las clases e interfaces del ecosistema `JpaRepository`.

1. Concepto y Beneficios de un ORM

Un **ORM (Object-Relational Mapping)** es una técnica de programación que permite traducir la estructura orientada a objetos de un lenguaje (Java, C#, Python) al modelo relacional de las bases de datos (tablas, filas, columnas y claves foráneas).

- **Productividad:** Elimina el código repetitivo JDBC/SQL (boilerplate code).
- **Abstracción:** Desconecta la lógica de negocio del motor de base de datos específico (fácil migración entre MySQL, PostgreSQL, Oracle).
- **Seguridad:** Previene la inyección SQL al emplear automáticamente consultas preparadas (Prepared Statements).
- **Mantenibilidad:** Permite manipular registros como objetos Java mediante métodos y tipado fuerte.

2. Arquitectura de Acceso a Datos en Java EE / Spring

La persistencia en Java sigue una jerarquía bien definida por capas de abstracción:

Capa	Tipo	Función Principal
1. Repositorio / DAO	Capa de Aplicación	Define métodos orientados al negocio (ej. `findByEmail`). Aisla la lógica de persistencia.
2. JPA (Java Persistence API)	Especificación estándar	Contrato de interfaces y anotaciones (`@Entity`, `@Id`). No contiene código ejecutable.
3. Hibernate	Implementación ORM	Librería concreta que implementa JPA. Mapea objetos, traduce consultas en SQL y gestiona cachés.
4. JDBC	API Base Java	Capa de bajo nivel que utiliza el driver de la BD para enviar sentencias SQL y procesar `ResultSet`.
5. Base de Datos	Motor SGBD	Almacenamiento persistente final (MySQL, MariaDB, PostgreSQL).

3. Referencia Completa de Anotaciones JPA y Hibernate

El documento incluye un compendio de las anotaciones más utilizadas en clases de entidad y sus atributos:

Anotación	Aplica a	Propósito / Función
`@Entity`	Clase	Marca la clase Java como una entidad mapeada a una tabla de base de datos.
`@Table(name="...")`	Clase	Especifica el nombre exacto de la tabla relacional (opcional si coincide).
`@Id`	Campo	Define el atributo como la clave primaria (Primary Key) de la entidad.
`@GeneratedValue`	Campo	Configura la estrategia de generación de la clave primaria (ej. `IDENTITY` para Auto-increment).
`@Column(...)`	Campo	Configura detalles de la columna (`nullable`, `length`, `unique`, `name`).
`@Lob`	Campo	Marca objetos grandes (textos extensos de tipo `TEXT` o datos binarios `BLOB`).
`@Transient`	Campo	Excluye un atributo de la persistencia (no se crea ni guarda en la base de datos).
`@Enumerated`	Campo (Enum)	Define cómo se mapea un Enum (recomendado `EnumType.STRING` para guardar el texto).

Anotación	Aplica a	Propósito / Función
`@ManyToOne` / `@OneToMany`	Campo (Relación)	Establece relaciones N:1 o 1:N entre entidades. Se complementa con `@JoinColumn`.
`@ManyToMany`	Campo (Relación)	Establece relaciones N:M, apoyándose en la anotación `@JoinTable`.
`@CreationTimestamp`	Campo	Anotación propia de Hibernate para auditar automáticamente la fecha/hora de inserción.
`@UpdateTimestamp`	Campo	Anotación propia de Hibernate para registrar automáticamente la última actualización.

4. Funcionalidades de `JpaRepository<T, ID>`

`JpaRepository` hereda de `CrudRepository` y `PagingAndSortingRepository`, ofreciendo una amplia gama de métodos out-of-the-box:

- **Operaciones CRUD Básicas:** `findAll()`, `findById(id)`, `save(entity)`, `deleteById(id)`, `existsById(id)`, `count()`.
- **Paginación y Ordenación:** `findAll(Sort sort)`, `findAll(Pageable pageable)` para procesar lotes paginados eficientemente.
- **Escritura Inmediata:** `flush()` y `saveAndFlush()` para forzar la sincronización con la base de datos en medio de una transacción.
- **Consultas Derivadas por Nombre de Método:** Spring Data traduce nombres de métodos en consultas SQL dinámicas de forma automática:
  - `findByAutor(String autor)` → `SELECT * FROM articulos WHERE autor = ?`
  - `findByPublicado(boolean pub)` → `SELECT * FROM articulos WHERE publicado = ?`
  - `findByTituloContainingIgnoreCase(String text)` → `SELECT * FROM articulos WHERE LOWER(titulo) LIKE LOWER(%text%)`

> **Buenas Prácticas Destacadas por la Documentación:**
> - Utilizar las API de tiempo modernas `LocalDate` y `LocalDateTime` de Java 8+ en lugar de `java.util.Date`.
> - Asegurar que toda entidad posea un constructor público o protegido sin argumentos (exigido por JPA).
> - Configurar explícitamente las estrategias de carga (`FetchType.LAZY` vs `FetchType.EAGER`) en relaciones para evitar problemas de rendimiento (como el problema de las N+1 consultas).